Montreal Backend SWE Plan

Document 1 of 6 — Overview, Daily Structure, and the Algorithm Path

The Goal

A signed offer for a Java backend software engineering role in Montreal, no later than **Friday, November 20, 2026**. You stay in Toronto and search Montreal employers remotely; you move only after the offer is signed.

The Commitment

Six hours a day, every day, from Friday September 4 through Friday November 20 — 78 days, about 468 hours. You write code every single day. If illness or emergency costs you a day, shift the whole plan forward one day rather than cramming two days into one.

The Standard Day

Block	Wk 1-4	Wk 5+	What happens
Algorithms	60 min	45 min	One named pattern per day, problems solved in a fixed order
Learn	150 min	120 min	New concepts, typed out by hand, never copy-pasted
Drills	(inside Learn/Build)		The numbered exercises in each day's gold panel — this is where retention actually happens
Build	120 min	135 min	Applying the day's concepts to your running project
Job search	—	60 min	From Day 29 onward. Follow Document 6.

The drills are the point. Reading about inheritance teaches you nothing durable. Writing eight small classes that use inheritance, breaking them, and fixing them is what makes it stick. Every day has a gold drill panel with numbered exercises. If you are short on time, cut the reading, never the drills.

The Algorithm Path — Why This Order

Patterns are learned in dependency order: each one reuses machinery from the one before it. Hashing before two pointers, two pointers before sliding window, stacks before trees, trees before graphs, recursion-by-way-of-backtracking before dynamic programming. Solving problems in a random order is the most common reason people grind hundreds of problems and still freeze in interviews.

Days	Pattern	Core idea you are internalizing
1-7	Arrays & Hashing	Trade memory for time: a HashMap turns an $O(n^2)$ scan into $O(n)$
8-9	Two Pointers	On sorted or symmetric data, two indices moving inward beat nested loops
10-11	Sliding Window	Maintain a valid window; expand right, shrink left, never re-scan
12-13	Stack	Last-in-first-out for matching, nesting, and 'next greater element'
14	Review	Redo one problem per pattern from a blank file, timed
15-17	Binary Search	Halve the search space — on arrays first, then on the answer space itself
18-20	Linked List	Pointer manipulation, dummy heads, and the fast/slow pointer trick
21	Review	Mixed set drawn from all patterns so far
22-25	Trees	Recursion over structure: DFS for depth questions, BFS for level questions

26	Tries	Prefix trees — the specialized structure interviewers use to test design ability
27-28	Heap / Priority Queue	Keep only what matters: top-K and streaming-median problems
29-32	Backtracking	Choose, explore, un-choose — the recursion skeleton behind all subset problems
33-35	Graphs	Grids and adjacency lists; DFS and BFS reused from trees
36-37	Advanced Graphs	Topological sort for dependencies, union-find for connectivity
38-41	1-D Dynamic Programming	Recursion with memory: recognize overlapping subproblems
42	Review	Mixed timed set
43-44	2-D Dynamic Programming	Grid and two-sequence DP built on the 1-D foundation
45-46	Greedy	When a local optimum provably gives the global one
47-48	Intervals	Sort by start or end, then merge or count — extremely common in real interviews
50-51	Math & Bit Manipulation	The small set worth knowing; not worth more than two days
52-78	Mixed / maintenance	Timed random sets, plus re-solving everything you previously failed

How to Work a LeetCode Problem

- Read the problem, then restate it out loud in your own words before writing anything.
- Give yourself 20 minutes of genuine attempt. Struggle is where the learning is — do not shortcut it.
- If stuck at 20 minutes, read the solution, close it, and re-implement from memory in a blank file.
- Write one line in your log: the pattern, the key insight, and what tripped you up.
- Any problem you could not solve unaided goes on a revisit list. Re-solve it 3 days later, then 10 days later.
- Never mark a problem 'done' because you read the solution. It is done when you can write it cold.

The 11-Week Structure

Week	Dates	Phase	What you have at the end
1	Sep 4 - Sep 10	Java syntax and OOP	Classes, inheritance, interfaces, exceptions — written from memory
2	Sep 11 - Sep 17	Collections and modern Java	A tested, Maven-built console application
3	Sep 18 - Sep 24	Spring Boot core	A layered REST API with validation and clean error handling
4	Sep 25 - Oct 1	PostgreSQL, JPA, testing	Migrations, relationships, and a real test suite
5	Oct 2 - Oct 8	Capstone + search launch	Capstone half-built; resume live; first applications out
6	Oct 9 - Oct 15	Ship the capstone	A deployed, documented, secured API with a public URL
7	Oct 16 - Oct 22	Microservices	A two-service event-driven system
8	Oct 23 - Oct 29	Interview depth	Concurrency, JVM, Spring internals, system design
9	Oct 30 - Nov 5	Interview intensive	Live processes and mock loops
10	Nov 6 - Nov 12	Convert	Final rounds and take-homes delivered
11	Nov 13 - Nov 20	Close	Offer negotiated and signed

The Six Documents

- **Document 1** (this one) — overview, daily structure, and the algorithm path.
- **Document 2** — Weeks 1-2, day by day with drills: Java fundamentals.
- **Document 3** — Weeks 3-4, day by day with drills: Spring Boot and databases.
- **Document 4** — Weeks 5-6, day by day with drills: the capstone project.
- **Document 5** — Weeks 7-11, day by day: microservices, interview depth, closing.
- **Document 6** — the Job Search Playbook: resume, French resume, targets, applications, interviews, negotiation.

Five Rules

- **Type everything.** Never copy-paste code you are learning. The slowness is the mechanism.
- **Do every drill.** They are numbered so you can tick them off. A day with reading but no drills is a wasted day.
- **One project, growing.** Depth over breadth — three substantial projects, not fifteen tutorials.
- **Commit daily.** A green contribution graph from September to November is itself evidence.
- **Never skip the job-search hour** once it starts on Day 29. Coding more instead feels productive and is the classic failure mode.

The Honest Risk

Your effort is not the risk — six hours a day for eleven weeks will make you technically ready. What you do not control is hiring speed. Montreal enterprise processes commonly run six to eight weeks from application to offer, which is exactly why applications start on October 2 rather than late October. If November 20 arrives with processes still live, that is a timing outcome, not a failed plan.